

Fact vs. Opinion: Text Classification using Natural Language Processing

Ceyhan Akdam and Juan Vasquez Guzman

Graz University of Technology, Graz, Austria

June 24, 2026

Abstract

This project explores different Natural Language Processing methods to classify short text segments as either "Facts" or "Opinions". Distinguishing between factual statements and subjective beliefs is quite a relevant task for applications like automated fact checking and social media moderation. Our work is split into two separate stages, using a dataset of 855 labeled sentences. In Stage 1, we use a traditional machine learning approach by extracting linguistic features, including TF-IDF vectors, part-of-speech (POS) density ratios, named entity recognition (NER) markers, and a custom Knowledge Base. Using these features to train a Logistic Regression classifier, we achieved a macro F1-score of 0.877 on our test set, which outperformed our basic rule-based baseline. In Stage 2, we implement an LLM approach using Llama-3.1-8B-Instant via the Groq API. We test six different prompting strategies, starting from a Zero-Shot setup up to a mixed approach, combining Retrieval Augmented Generation (RAG), Few-Shot examples, and Self Consistency majority voting. Our best performing pipeline was the full hybrid combination, which achieved a macro F1-score of 0.959. Finally, we applied said pipeline to predict labels for an unlabeled validation set of 222 samples. Our results conclude that while traditional machine learning is fast and computationally light, combining multiple prompt engineering methods with an LLM provides the best classification performance.

1 Introduction

Distinguishing objective factual statements from subjective opinions is an important task in natural language processing. A fact represents an objectively verifiable claim backed by data or evidence, while an opinion represents a personal belief or interpretation that cannot be universally verified. Creating an automated system to separate these text types is topical for fields like automated fact checking and social media moderation.

The goal of this project is to implement and evaluate different text classification workflows to separate short text segments into facts or opinions. We were provided with a labeled training dataset of 855 sentences (443 facts and 412 opinions) along with an unlabeled validation dataset of 222 samples for final testing.

To solve this assignment, we implement two independent classification approaches. Stage 1 explores a traditional machine learning approach by engineering linguistic features, such as TF-IDF vectors, part-of-speech (POS) ratios, and named entities (NER) to train a Logistic Regression classifier against a simple rule-based baseline. On our other strategy, Stage 2 evaluates an LLM approach using Llama-3.1-8B-Instant via the Groq API. This second stage benchmarks prompting methods ranging from basic Zero-Shot inference to a hybrid combination of Retrieval Augmented Generation (RAG), Few-Shot examples, and Self Consistency majority voting. This parallel structure allowed us to compare the performance and efficiency of feature-based learning against modern genera-

tive strategies on the same test split.

2 Related Work

Using traditional linguistic features has a long history in text classification tasks. Early research showed that certain part-of-speech (POS) patterns and word frequencies relate closely to subjective language and opinions (Wiebe et al., 2004). Recent studies have shown that looking at specific grammatical structures and function words can serve as stable indicators when separating opinion pieces from factual text (Krüger et al., 2025).

With the introduction of LLMs, text classification research has shifted toward prompt engineering strategies. Initial work showed that pre-trained language models can complete classification tasks successfully when given specific input-output examples directly inside the prompt (Brown et al., 2020). Also, research has analyzed the boundaries of how models follow direct instructions without examples, showing that small structural changes in system prompts considerably affect how well an LLM reasons and classifies information (Kojima et al., 2022).

With ideas based on these sources, our project evaluates both stages by comparing traditional NLP feature engineering against different LLM prompt strategies for classification.

3 Materials and Methods

3.1 Dataset

The training dataset consists of 855 labeled text entries, split between 443 facts (51.8%) and 412 opinions (48.2%). To evaluate our models properly, we applied an 80/20 stratified train-test split using a fixed random state ($random_state = 42$) for reproducibility. This splits our total sample pool ($N = 855$) into a training set (N_{train}) and a test set (N_{test}):

$$N_{train} = 855 \times 0.80 = 684 \quad (1)$$

$$N_{test} = 855 \times 0.20 = 171 \quad (2)$$

Additionally, a separate unlabeled validation dataset contains 222 text samples for the final pipeline testing.

3.2 Stage 1: Traditional NLP

For our first approach, we constructed a supervised machine learning pipeline by stacking four linguistic representations to create a feature matrix:

- **Knowledge Base (KB):** A 21-dimensional vector of manual opinion markers (e.g., "i think") and fact markers (e.g., "percent").
- **TF-IDF:** A unigram vectorizer with a vocabulary cap of 5,000 terms to act as our primary lexical signal.
- **POS Ratios:** Text segments parsed via spaCy to calculate the frequency proportions of part-of-speech tags.
- **NER Ratios:** Frequencies of extracted named entity types normalized by the total token length.

We created a simple rule-based baseline that counts and compares the raw KB keywords. For this stage, the combined feature matrix was passed to a Logistic Regression classifier ($max_iter = 1500$).

3.3 Stage 2: LLM-Based Classification

For our second approach, we evaluate LLM capabilities using the Llama-3.1-8B-Instant model via the Groq API. To ensure reproducible outputs across all evaluations, the API temperature parameter was fixed at 0. We benchmarked six prompting workflows on our 171 test samples:

1. **Zero-Shot (Baseline):** A basic system prompt defining fact and opinion.
2. **Few-Shot Learning:** Attaching four training examples directly into the prompt context.
3. **RAG:** Running a local keyword lookup against our KB list and automatically injecting matched terms as system prompt hints.
4. **RAG + Few-Shot:** Stacking both the contextual keyword hints and the four examples.

5. **Self-Consistency:** Running the Zero-Shot prompt through three parallel API calls and taking a majority vote to reduce random model variance.
6. **RAG + Few-Shot + Self-Consistency:** Our fully combined pipeline running the hybrid prompt across three runs to obtain the final majority vote label.

4 Results

Our evaluation is split into two experimental procedures, both executed against the same hidden test split ($N_{test} = 171$). Stage 1 tracks the performance of traditional feature-engineered classifiers, while Stage 2 tracks LLM configurations. The performance across these stages can be seen in detail in Figure 1 and Figure 2, respectively.

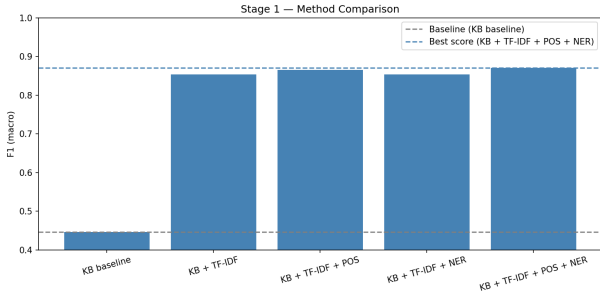


Figure 1: Stage 1: Feature configuration performance across classification benchmarks.

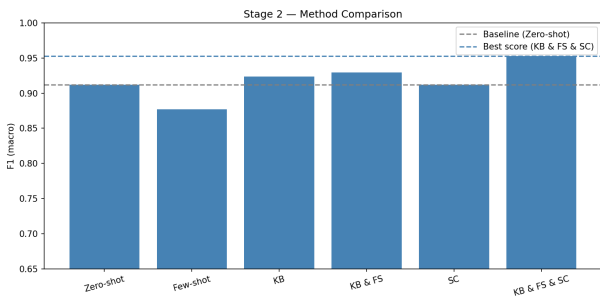


Figure 2: Stage 2: Prompting performance across Llama-3.1-8B-Instant tasks.

4.1 Stage 1: Performance Evaluation

The performance of the traditional machine learning approach is detailed in Table 1. The

baseline model relying only on raw keyword counts from the manual Knowledge Base (KB) resulted in a macro F_1 score of 0.4462, limited by a low recall on opinion text.

Applying a unigram TF-IDF vectorizer improved classification performance, pushing the macro F_1 score to 0.8535. Sequentially expanding the feature space resulted in noteworthy behaviors: adding syntactic properties (POS ratios) slightly optimized the macro score to 0.8654, while including named entities (NER ratios) made no difference at all, matching the isolated TF-IDF metric exactly. This specific NER behavior is analyzed in Section 5. In the end, combining all engineered dimensions together achieved the peak Stage 1 metric with a final macro F_1 score of 0.8713.

Table 1: Stage 1 Classification Macro F_1 Results

Feature Configuration	Macro F_1
KB Baseline (Rule-Based)	0.4462
KB + TF-IDF	0.8535
KB + TF-IDF + POS	0.8654
KB + TF-IDF + NER	0.8535
KB + TF-IDF + POS + NER	0.8713

4.2 Stage 2: Performance Evaluation

The prompt-engineering benchmarks through the Llama-3.1-8B-Instant model are summarized in Table 2. Even in a basic Zero-Shot state, the model showcased great contextual understanding of text styling, outperforming Stage 1’s best supervised setup with a macro F_1 score of 0.912.

Surprisingly, the standalone Few-Shot approach caused a performance decrease down to 0.877. On the other hand, implementing a Knowledge Base keyword lookup within the prompt context (RAG) stabilized classification to an F_1 score of 0.923.

The best performance optimization method was realized by establishing a mixed pipeline. Combining RAG, Few-Shot, and Self Consistency majority votes (RAG + FS + SC) resulted in the highest overall pipeline score for this project, providing a final macro F_1 score of 0.953.

Table 2: Stage 2 Prompt Engineering Macro F_1 Comparison

Prompting Workflow	Macro F_1
Zero-Shot (Baseline)	0.912
Few-Shot Learning	0.877
Knowledge-Based Learning (RAG)	0.923
Self-Consistency Learning (SC)	0.912
Knowledge-Based & Few-Shot	0.930
RAG + Few-Shot + SC (Hybrid)	0.953

5 Discussion

The route of our testing highlights different trade-offs between feature engineering and prompting frameworks. In Stage 1, an important discovery was that adding NER to the *KB* + *TF-IDF* mix made no performance difference ($F_1 = 0.8535$). This lead us to perform some research and we believe that our TF-IDF vectorizer already models vocabulary frequencies for entity tokens (e.g., proper names or locations). On top of that, named entities seem to not have discriminative power for this task because specific names and dates appear equally in objective facts and subjective opinions. On the other hand, POS densities showcase unique structural information by capturing stylistic patterns (such as heavy adjective and pronoun usage in opinions), which pushed our score up to 0.8713.

In Stage 2, we learned that generative models already possess strong native text-style awareness, as Llama-3.1-8B-Instant outperformed our best Stage 1 supervised setup using only a basic Zero-Shot instruction ($F_1 = 0.912$). That being said, isolating Few-Shot learning examples caused an unexpected performance drop down to 0.877. This could suggests that providing a small set of only four training examples introduces an initial recency bias, constraining the model’s pre-trained contextual background. We can conclude that for subjective text tasks, standalone few-shot prompting should be avoided. Instead, a mix of frameworks that pair RAG with Self Consistency provide the most solid setup, pushing our peak score of 0.953.

6 Conclusion

In this project, we evaluated and compared traditional supervised machine learning pipelines against various LLM prompt-engineering methods to classify text entries as either facts or opinions. Across both stages, our experiments highlighted technical trade-offs between manual feature engineering and generative learning.

Our main observation from Stage 1 is that traditional classifiers are sensitive to feature choices. While extracting POS ratios successfully captured structural style patterns (e.g., heavy adjectives and pronoun usage in opinions), adding NER tags introduced feature redundancy against our TF-IDF baseline. This showcased that word frequencies already capture entity data, meaning manual feature stacking should be done carefully to avoid multicollinearity. In the end, our final mixed model reached a peak macro F_1 score of 0.8713.

For Stage 2, the generative track highlighted that while Llama-3.1-8B-Instant has great native text understanding to begin with, it is quite sensitive to prompt configurations. Isolating few-shot examples seems to have introduced a recency bias that dropped performance down to a F_1 score of 0.877. The biggest takeaway from this section was that standalone prompting should be avoided for subjective tasks; instead, combining contextual hints via RAG with majority-voting via Self-Consistency provided the best pipeline, pushing our highest overall score of 0.953.

For future projects, this assignment could be expanded by testing our pipelines on more varied datasets that include mixed text formats, such as data sets created specifically from news articles or online forums. This would help evaluate how well our mixed feature matrices and RAG context prompts handle complex sentences where facts and opinions are blended together.

Limitations

There were a few limitations in our current implementation. First, our entire pipeline is strictly limited to English text; It cannot handle multi-lingual data formats without translating the inputs first.

Second, a considerable practical issue we faced during Stage 2 was working around the rate and token limits of using a free-tier cloud service. To execute our entire test script in a single run without reaching rate blocks, we had to implement a setup that automatically switches through two separate Groq API keys. While running a local open-source model would have given us a slower processing speed, it definitely would have been a better option for a task as large as Stage 2 because it would completely remove the token limits we faced when accessing the external Groq API.

Ethical Considerations

There are a few ethical considerations related to both the technical resources used and the resulting output of our classification system. First, our implementation required considerable computational resources during Stage 2 due to frequent API calls to the Groq platform. Because we implemented prompting frameworks like Self Consistency, which evaluates multiple output branches to determine a majority vote, our pipeline processed a high volume of tokens. This dependency on cloud hosted LLMs implies an indirect environmental cost by the nature of data center power consumption.

Second, a possible ethical risk revolves around how labeling content as a "Fact" or an "Opinion" could potentially cause harm to users if implemented in real world systems. Mislabeling a personal opinion as an objective fact may lead to the spread of misinformation. On the other hand, mislabeling an objective fact as an opinion could undermine trust in valid scientific or journalistic reports. This is why a system like the one we implemented should be used as a support tool instead of an automated moderator.

Acknowledgments

We would like to thank Professor Roman Kern for his guidance and excellent lectures throughout this course. We are also grateful to the supportive teaching staff, including Ratko Savic, Yoram Dieter, and others, for providing their support in the lectures and insight during interviews.

Additionally, we are very thankful to our colleague Luis Garrido, a previous classmate from our bachelors and currently a Data Engineer whose insights were very helpful when approaching our Stage 2 prompt-engineering script and managing API pipelines.

7 Contributions

Table 3: List of Team Contributions per Member

Task	Ceyhan	Juan
Dataset analysis	50%	50%
Stage 1	30%	70%
Stage 2	80%	20%
Report writing	40%	60%

References

- Tom Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 2020.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*, 35, 2022.
- Fritz Krüger et al. Function words as stable features for opinion articles classification. In *Proceedings of KONVENS*, 2025.
- Janyce Wiebe, Theresa Wilson, and Claire Cardie. Annotating expressions of opinions and emotions in language. *Language Resources and Evaluation*, 39(2), 2004.

A Appendix - Overview

We initially planned to implement Chain of Thought (CoT) to increase the model's performance. However, upon realizing that our current model was already reaching an F1 score close to 100%, we decided to not implement CoT. We made this decision to avoid the risk of overfitting and to decrease the computational cost of the script.

In addition to this, during Stage 1 we initially evaluated more combinations for our mixed models, such as testing the performance of an isolated *TF-IDF + POS* feature setup. However, after visualizing our final results, we decided to only keep the relevant configurations in our main report. Since the project guidelines required working off of a Knowledge Base, configurations that did not implement the manual Knowledge Base baseline were excluded.

B Appendix - Usage of AI

During this project, we made use of ChatGPT to support the following development steps:

- In Stage 2, to discover efficient methods for accessing Groq's API and ideas on how to implement the two-key switching structure to handle token limits.
- To better structure the report in LaTeX format, ensuring a professional layout and correcting structural mistakes.

Every implementation choice, final code structure, and metric evaluation was managed manually by the team. The AI model served only as a tool for brainstorming and final documentation editing.

Appendix C - Model Parameters

Logistic Regression Classifier (Stage 1)

- **Vectorizer:** Unigram TF-IDF Vectorizer
- **Feature Dimensions:** Manual Knowledge Base, POS ratios, and NER ratios

- **Maximum Iterations:** 1500
- **Random State:** 42
- **Train-Test Split:** 80/20 ratio

Llama-3.1-8B-Instant via Groq API (Stage 2)

- **Model:** Llama-3.1-8B-Instant
- **Temperature:** 0 (for deterministic Zero-Shot and Few-Shot classification)
- **Few-Shot Exemplars:** 4 training examples (2 facts, 2 opinions)
- **Self-Consistency Paths:** 3 generation chains for Self Consistency
- **API Rate Optimization:** 2 API keys configured for automated script rotation

Appendix D - Figures

This appendix provides a better visualization of our graphs from Stage 1 and Stage 2.

Stage 1: Feature Matrix Evaluation

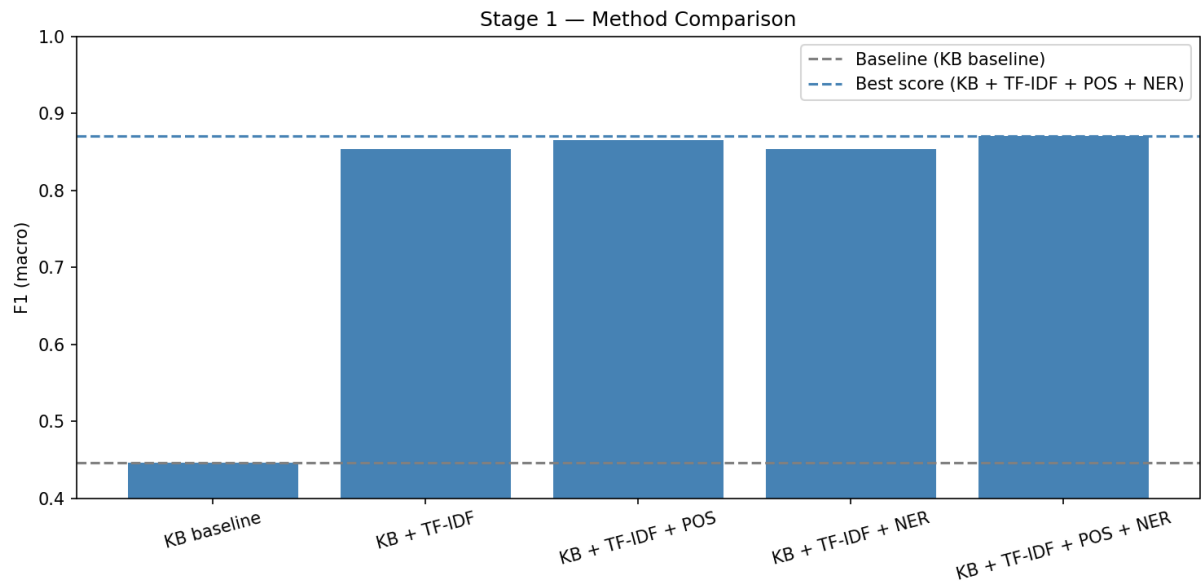


Figure 3: Detailed performance breakdown and evaluation metrics for the Stage 1 supervised learning models.

Stage 2: Prompting Framework Comparisons

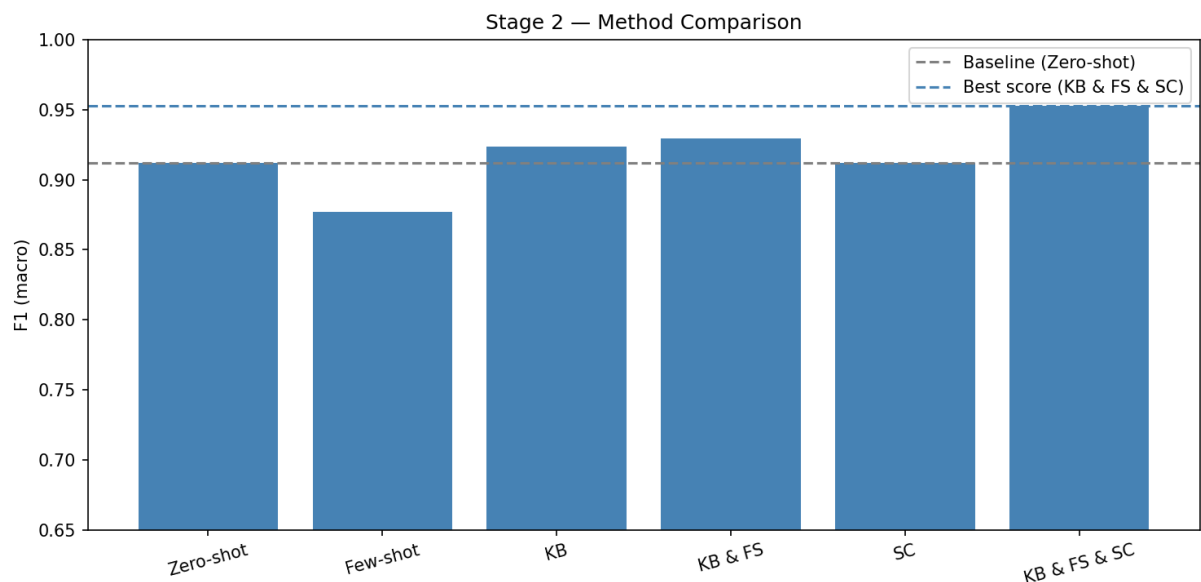


Figure 4: Performance validation across the baseline, Zero-Shot, Few-Shot, and final RAG with Self-Consistency pipelines.